

No Man's Land

A custom SystemVerilog GPU and game on the DE1-SoC

Rohit Biswas Kambinachi Obioha Nicola Paparella

CSEE 4840 — Embedded System Design — Spring 2026

Instructor: Prof. Stephen Edwards

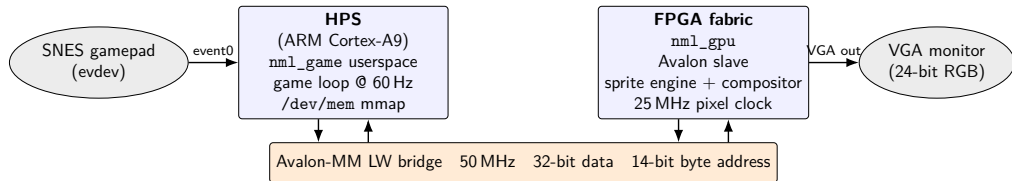
The system, running

PHOTO GOES HERE

`submission/img/01_system_running.jpg`

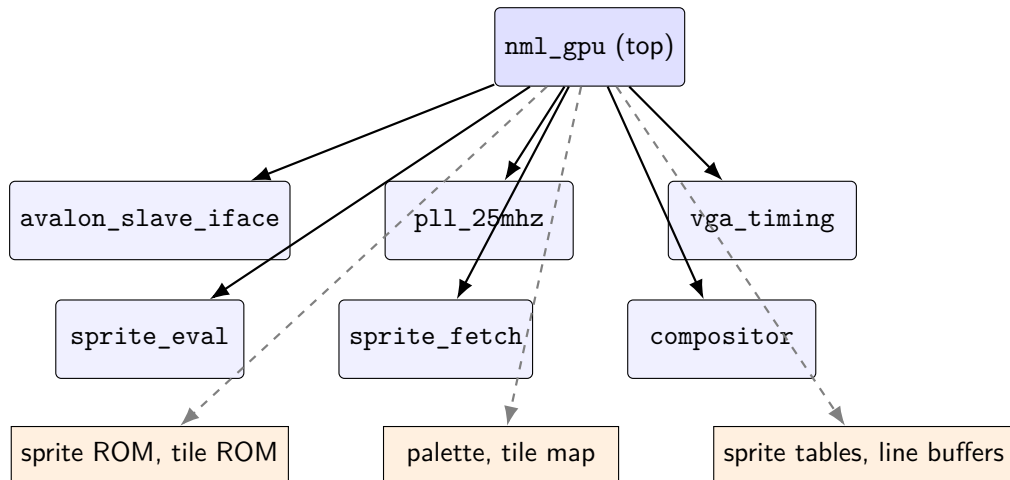
Board • VGA monitor • SNES gamepad, running

System block



HPS writes shadow state; FPGA latches at VSYNC; pixels always stream.

What's inside nml_gpu

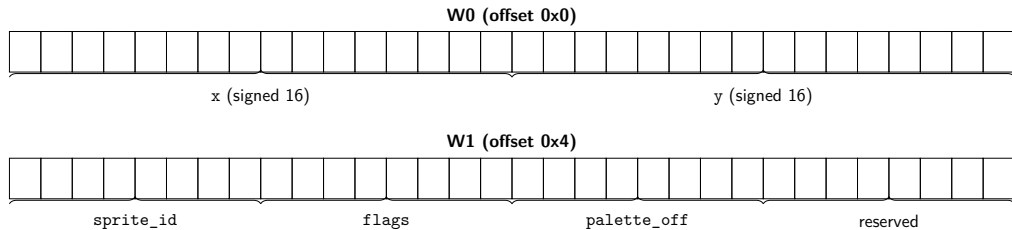


One 50 MHz domain (Avalon) + one 25 MHz domain (pixel). All RAMs/ROMs inferred as M10K.

Avalon register map — 16 KB window at 0xFF200000

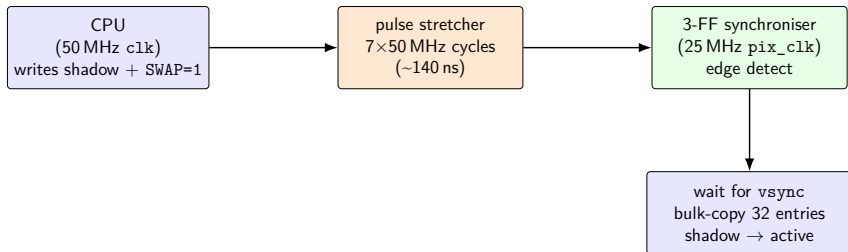
Offset	Name	Layout
0x0000	CTRL	[0]=ENABLE, [1]=SWAP (auto-clear), [2]=HUD_ON
0x0004	STATUS	[0]=VBLANK, [1]=SWAP_PENDING
0x0008	BG_SCROLL	[15:0]=scroll_x, [31:16]=scroll_y (signed)
0x000C	IRQ_MASK	[0]=VBLANK IRQ enable
0x0010	PLAYER_POS	[15:0]=px, [31:16]=py
0x0014	PLAYER_STATS	[7:0]=hp, [15:8]=wave (BCD), [31:16]=level
0x0018	SCORE	[23:0]=6-digit BCD score
0x001C	KILL_COUNT	32-bit kills
0x0020	HUD_AUX	[7:0]=ammo BCD, [11:8]=art, [15:12]=gas
0x0100-0x01FF	Sprite table	32 entries × 8 B
0x0400-0x07FF	Palette	256 entries × 4 B (RGB888)
0x1000-0x22BF	Tile map	80 cols × 60 rows, 1 byte/tile

Sprite-table entry — 8 bytes / 64 bits



flags byte: [2] hflip [3] vflip [6:4] priority (0 = highest) [7] ACTIVE

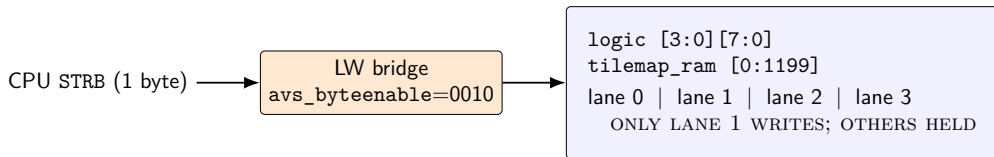
Frame commit — shadow → active at VSYNC



CPU then polls `STATUS.SWAP_PENDING`;
clears one cycle after the bulk copy.

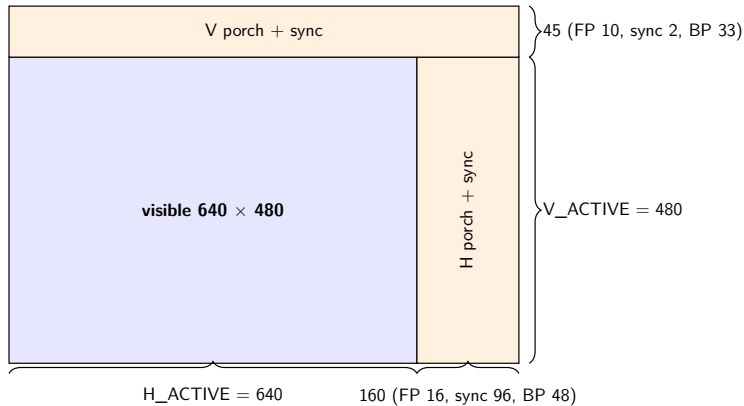
One 20 ns Avalon pulse would slip past the 40 ns pixel clock. Widen to 140 ns and the CDC is deterministic.

Tile-map byte writes — packed [3:0] [7:0]



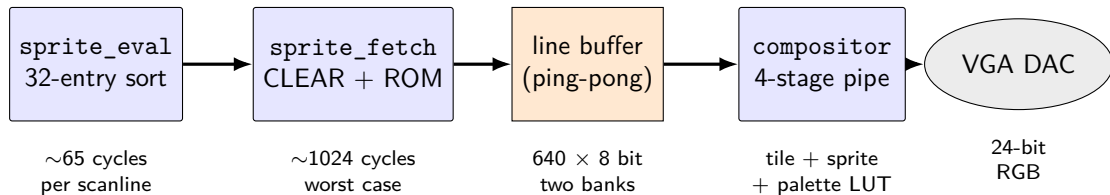
Why packed? Byte-organised storage forced bridge RMW to clobber 3 lanes per word. Packed inference uses M10K byteena_a: one lane updates, the other three are untouched.

VGA timing — 640×480 @ 60 Hz, 25 MHz dot



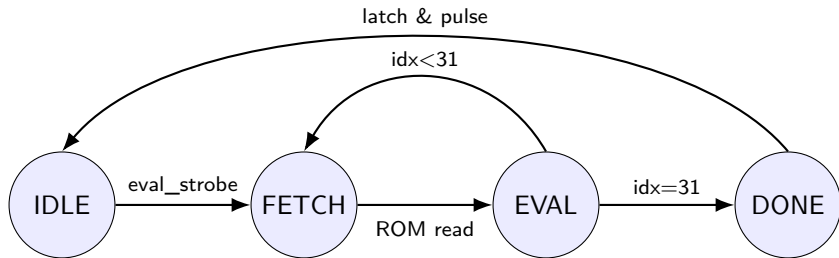
One frame = 800×525 dot cycles. HBLANK is the 160-cycle window we use to evaluate sprites for the next scanline.

Pixel pipeline — sprite engine + compositor



Eval runs inside HBLANK (160 cycles). Fetch spans HBLANK + visible (800 cycles). Line buffers are double-buffered, so fetch lags read by one scanline. Compositor never stalls.

sprite_eval — per-scanline priority sort



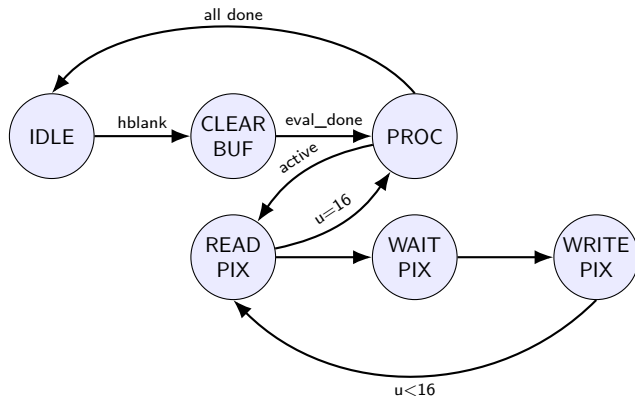
Per sprite (1 cycle in EVAL):

`intersects = active && y ≤ next_scanline < y+16`

insertion-sort into 8 priority-ordered slots (all 8 compares in parallel)

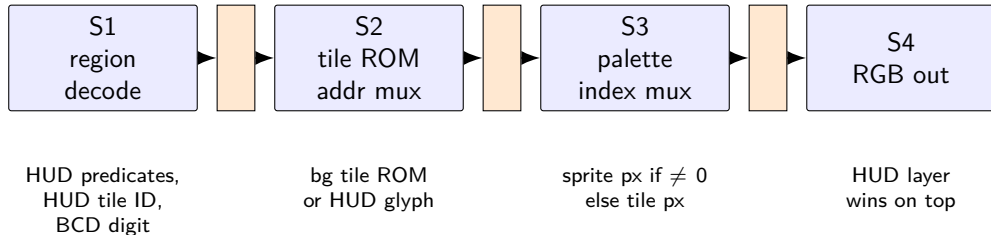
Total: 32 fetch+eval cycles + 1 DONE = ~65 cycles. Fits easily in 160-cycle HBLANK.

sprite_fetch — ROM read + line-buffer fill



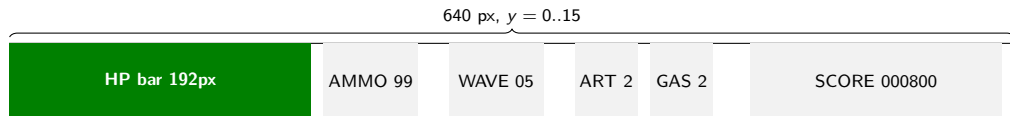
Worst-case cycle budget per scanline: $\text{CLEAR_BUF (640)} + 8 \text{ sprites} \times 48 \text{ (384)} \approx 1024 \text{ cycles}$.
HBLANK alone is 160; HBLANK + visible is **800**. Fetch lives one line behind the read, so the ping-pong line buffers absorb the latency.

Compositor — 4-stage pipeline



Latches between stages absorb the 1-cycle latency of tile ROM and palette M10K reads. Never stalls.

HUD strip — top 16 pixels of every frame



HP bar: 2 px per HP, clamped to 192. Glyphs: 8×8 from tile ROM, slots 16–41 (A–Z), 48–57 (0–9). Scores, ammo, wave packed BCD in registers.

HUD — on the monitor

PHOTO GOES HERE

`submission/img/02_hud_closeup.jpg`

Closeup of the HUD strip on the VGA monitor

Memory inventory (all inferred M10K)

Memory	Shape	Init	Bits
sprite ROM	$16,384 \times 8$	\$readmemh	131,072
tile ROM	$4,096 \times 8$	\$readmemh	32,768
tile map	$1,200 \times 32$ (packed 4×8)	blank (HPS writes)	38,400
palette	256×24	\$readmemh + HPS overrides	6,144
sprite table act.	32×64	via VSYNC bulk copy	2,048
sprite table shd.	32×64	HPS writes	2,048
line buffer A	640×8	cleared per HBLANK	5,120
line buffer B	640×8	cleared per HBLANK	5,120
Total			86,144 bits

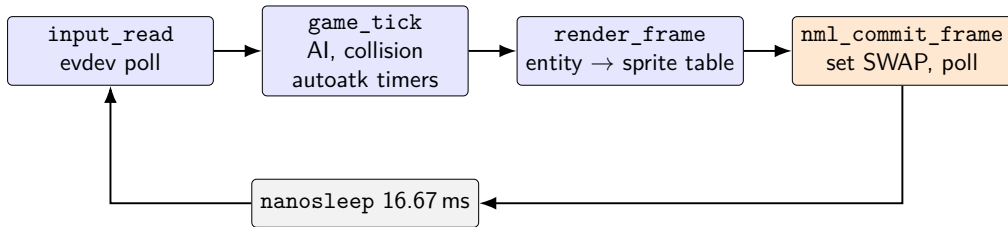
14 M10K blocks out of 397.

Resource utilisation — Cyclone V 5CSEMA5F31C6

Resource	Used	Available	%
ALMs	205	32,070	< 1%
Registers	365	—	—
Pins	54	457	12%
Block memory bits	86,144	4,065,280	2%
M10K blocks	14	397	4%
DSP blocks	0	87	0%

Compile time: 63 s wall clock (Analysis 13 s, Fitter 30 s, Asm 12 s, STA 8 s).

Software architecture — 60 Hz game loop on the ARM



`game_t` holds a 64-entity pool. State machine: `PLAYING` ↔ `LEVELUP` → `GAMEOVER`.
Mortar is timer-driven; artillery + gas fire from L/R shoulders.

What actually works

PHOTO GOES HERE

`submission/img/03_gameplay.jpg`

Live gameplay on the VGA monitor

640×480 @ 60 Hz • tile-mapped battlefield • 32 hardware sprites • 256-entry palette

HUD: HP bar, AMMO, WAVE, ART, GAS, SCORE (BCD)

SNES gamepad via evdev; 60 Hz HPS game loop; frame-commit at VSYNC

Mortar, artillery, gas clouds, wave spawner, level-up, ammo drops, game-over